Does Sample-Level Model Fusion Age Well? A Deployment-Faithful Evaluation of TimeFuse

Muhammad Imad Khan
BS Data Science, Ghulam Ishaq Khan Institute (GIKI), Pakistan

Technical report, November 2026. Code and results: github.com/imadkhan4478/timefuse-deployment-eval

Abstract

TimeFuse learns, for every input window, how to weight the forecasts of several base models. Its fusor is fitted once on the validation period and then evaluated on a fixed test split. Deployment differs in two ways: the fusor keeps running long after the period it was fitted on, and the practitioner's realistic alternative is the single model that looked best on validation, not the model that turns out best in hindsight. I replicate TimeFuse on ETTh1 and ETTh2 (MSE 0.368 and 0.282 against the reported 0.367 and 0.280) and re-evaluate it under three deployment-faithful protocols: evaluation on four months that the standard split never uses, monthly refitting of the fusor, and fusors deliberately fitted on stale windows. Three findings follow. (1) TimeFuse ages well: a fusor up to four months staler loses at most 0.7% MSE, and monthly refitting changes error by at most 1.1%, never significantly. (2) Its practical value is larger than the benchmark comparison suggests: against the validation-selected model, TimeFuse cuts MSE by 7.1% on the ETTh1 test months and 7.2% on the unused months (Diebold-Mariano $p < 0.001$), compared with 1.8% against the hindsight-best model, because on ETTh1 the identity of the best single model changes every month. (3) With the six-model pool of the public repository, an equal-weight average of the same models matches TimeFuse, and is in fact 0.2% to 1.4% more accurate in all four settings (significantly in three). The learned weights stay close to uniform. In this setting, fusion's value comes mainly from insurance against model-selection error rather than per-sample adaptation, which raises a concrete question about when learned weighting pays off.

1 Motivation

No single forecasting architecture wins everywhere, and TimeFuse (Liu et al., 2025) turns that observation into a method. It extracts meta-features from each input window (statistical, temporal, spectral and cross-channel summaries), maps them through one linear layer and a softmax to a weight per base model, and outputs the weighted sum of base forecasts. The fusor is trained on held-out validation predictions and reported state-of-the-art results across long-term, short-term and electricity-price forecasting.

My own prior work on air-quality forecasting in Lahore taught me that the evaluation protocol can change which model looks best: a gradient-boosting model that ranked first under a random split fell behind once I evaluated it the way it would actually be deployed. That experience led to the question behind this study. A sample-level fusor is itself a model selected and fitted on one period. Does what it learns about *which model to trust when* survive as the series moves on, and how should its gain be measured when the practitioner cannot know the best model in advance?

The benchmark protocol used by TimeFuse and most of the field leaves both questions open. The standard ETT split is fixed (12/4/4 months), the fusor is fitted once on the four validation months, and the last four months of each ETT series are never used. The headline comparison is against the best individual model on the test set, which is chosen with hindsight. This study keeps TimeFuse exactly as released and changes only the evaluation.

2 Setup

Faithful replication. I use the authors' public code and its default configuration: ETTh1 and ETTh2, input length 96, horizon 96, and six base models, DLinear, PatchTST, TimesNet, PAttn, TimeMixer and TimeXer (Zeng et al., 2023; Nie et al., 2023; Wu et al., 2023; Tan et al., 2024; Wang et al., 2024a; Wang et al., 2024b). These use the official Time-Series-Library hyperparameters that TimeFuse loads. Base models are trained once (seed 2021) on the first 12 months with early stopping on validation. One fusor is trained jointly on both datasets, as in the released notebook: Huber loss ($\beta = 0.01$), Adam (learning rate 5×10^{-4}), 5 epochs and batch size 64, with 22 meta-features as implemented in the code. Every fusor result is averaged over five fusor seeds. Errors are computed on standardized data, as in the paper.

Extended forecast stream. I run every trained base model at every forecast origin from the start of validation to the end of the data: months 13 to 24, or 8,685 windows per dataset. Months 17 to 20 are the benchmark test split, and months 21 to 24 are never used by the standard protocol. Base models are never retrained, so every forecast after month 12 is out of sample for them.

3 Protocols

- **A (paper protocol).** Fit the fusor on validation (months 13 to 16), evaluate on the test split (months 17 to 20).
- **A+ (unused months).** Same fusor, evaluated on months 21 to 24.
- **B (monthly refit).** At the start of each month, refit the fusor on the most recent four months of windows whose targets are already observed, then forecast that month. The comparator is the rolling single model, chosen each month on the same four-month window.
- **C (staleness curve).** For each evaluation month from 21 to 24, fit fusors on four-month windows that end 0 to 4 months before that month. Holding the evaluation month fixed isolates the effect of the fusor's age from month-to-month difficulty.

Comparators and statistics. I compare TimeFuse with (i) the single model a practitioner would deploy, the one with the lowest validation MSE; (ii) the hindsight-best single model on the evaluation period, which is the paper's comparator; and (iii) an equal-weight average of the same six forecasts. Differences in per-window MSE are tested with the Diebold-Mariano test with the Harvey-Leybourne-Newbold small-sample correction and a Newey-West variance at lag 95, because consecutive 96-step forecasts overlap. 95% intervals come from a moving-block bootstrap with one-week blocks and 2,000 resamples (Diebold & Mariano, 1995; Harvey et al., 1997; Künsch, 1989).

4 Results

4.1 The replication matches the paper

TimeFuse and the best single model differ from the published values by 0.0012 to 0.0021. One detail deserves attention before any re-evaluation. On ETTh1, validation selects DLinear, which then ranks last of the six on the test split. On ETTh2, validation selects TimeXer, which is also the best on test. Validation-based selection is therefore unreliable on one dataset and reliable on the other.

MSE	DLinear	PatchTST	TimesNet	PAttn	TimeMixer	TimeXer	Equal avg.	TimeFuse	Paper
ETTh1 test	0.396	0.379	0.389	0.385	**0.375**	0.381	0.367	0.368	0.367
ETTh2 test	0.341	0.299	0.337	0.299	0.292	**0.286**	0.279	0.282	0.280
ETTh1 unused	0.552	0.519	0.587	0.524	**0.518**	0.521	0.509	0.512	n/a
ETTh2 unused	0.204	0.194	0.213	0.187	**0.185**	0.186	0.175	0.177	n/a

Table 1. MSE on standardized data, horizon 96. Bold marks the hindsight-best single model. The paper's "best individual model" is 0.377 on ETTh1 and 0.285 on ETTh2, against 0.375 and 0.286 here. "Test" is months 17 to 20 and "unused" is months 21 to 24. TimeFuse is the mean of five fusor seeds (standard deviation at most 0.002); "Paper" is the published TimeFuse result. Validation selects DLinear on ETTh1 and TimeXer on ETTh2.

4.2 TimeFuse ages well

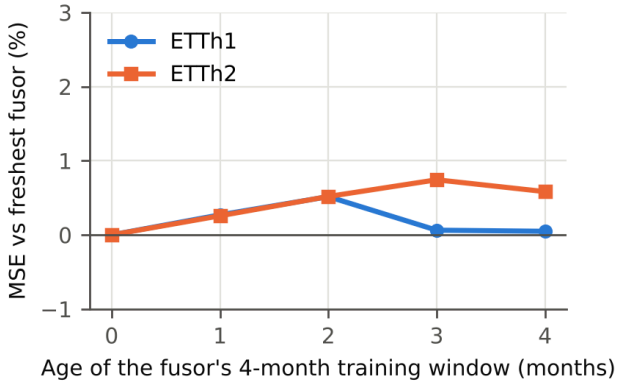


Figure 1. Protocol C. MSE of fusors fitted on older four-month windows, relative to the freshest fusor, averaged over evaluation months 21 to 24.

The expected failure, a fusor whose learned preferences go stale, does not appear. A fusor fitted on a window two to four months older loses at most 0.74% MSE (ETTh2, three months) and 0.52% at most on ETTh1 (Figure 1). Monthly refitting (Protocol B) improves MSE by −0.2% and 0.0% on ETTh1 and by 1.1% and 1.0% on ETTh2 (test and unused months respectively), and none of these differences is significant (all Diebold-Mariano $p \geq 0.10$). For a practitioner, the fusor can be fitted once and left alone for at least eight months on these series.

4.3 Its deployment value is larger than the benchmark shows

Against the model a practitioner would actually deploy, TimeFuse's gain on ETTh1 is about four times its gain against the hindsight-best model (7.1% vs 1.8%), and the gap persists on the unused months (7.2%). The reason is visible in Figure 3: on ETTh1 the best single model changes every month, and five different architectures take a turn at the top. Under that much rank churn, choosing a single model is a bet that usually loses, and any combination hedges it. On ETTh2, where validation happens to pick the right model, the gain is smaller and significant only on the unused months. The advantage also survives a stronger baseline: when the single model is re-selected every month on the latest four months (Protocol B), TimeFuse is still 6.6% better on the ETTh1 test months ($p < 0.001$) and 5.7% better on the ETTh2 unused months ($p = 0.002$).

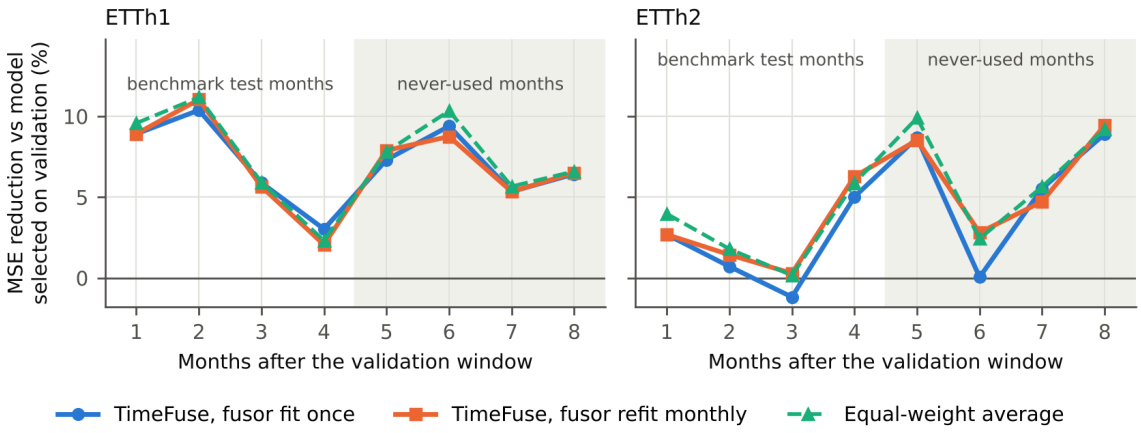


Figure 2. Monthly MSE reduction relative to the single model selected on validation. Shaded: months 21 to 24, which the standard split never evaluates. The refitted fusor and the equal-weight average beat the validation-selected model in all 16 dataset-months; the fixed fusor does in 15.

TimeFuse vs.	ETTh1 test		ETTh1 unused		ETTh2 test		ETTh2 unused	
	gain	p	gain	p	gain	p	gain	p
Validation-selected single model	7.1%	<.001	7.2%	<.001	1.7%	.47	4.8%	.006
Hindsight-best single model	1.8%	.03	1.1%	.14	1.7%	.47	4.1%	.08
Equal-weight average	−0.2%	.18	−0.6%	.03	−1.1%	.008	−1.4%	.007

Table 2. Relative MSE reduction of TimeFuse (positive means TimeFuse is better) with Diebold-Mariano (HLN) p-values. Block-bootstrap 95% intervals agree in sign with every result significant at the 0.05 level.

4.4 An equal-weight average matches it here

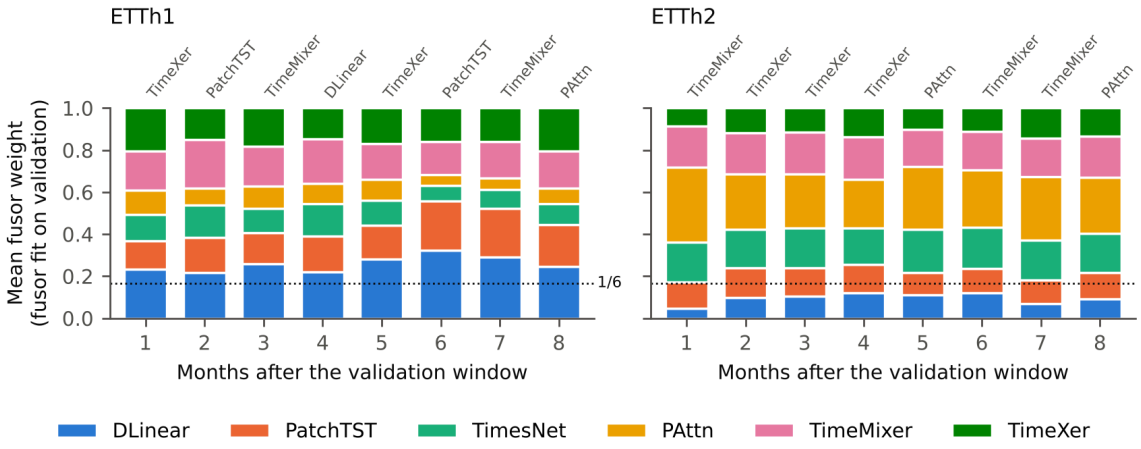


Figure 3. Mean fusor weight per base model by month (fusor fitted on validation). The label above each bar names the hindsight-best single model of that month. The dotted line marks uniform weight (1/6).

The third row of Table 2 is the finding I did not expect. With the six-model pool of the public configuration, an equal-weight average is as accurate as TimeFuse: it is 0.2% to 1.4% better in all four settings, and significantly so in three. Figure 3 suggests why. The learned monthly weights stay between 0.05 and 0.36, close to uniform, and barely move from month to month, which also explains the robustness in Section 4.2. The fusor also inherits the validation period's preferences. On ETTh1 it gives DLinear the largest weight in seven of eight months, although DLinear ranks fifth or sixth in

seven of the eight months. Blending dilutes that inherited error, but per-sample weighting does not correct it.

This does not contradict the paper, which reports TimeFuse ahead of mean and median ensembles with validation-based top-k filtering on a 13-model zoo that includes much weaker models such as Informer (MSE above 1.0 on ETTh1, averaged over horizons). In such a pool, learning to switch weak models off is valuable. The two results together point to a dependence on the pool that the benchmark does not yet measure: when the candidates are all competitive, most of the benefit is diversification, which equal weights already capture. This echoes the "forecast combination puzzle" in econometrics, where simple averages often beat estimated weights (Stock & Watson, 2004; Claeskens et al., 2016).

5 What this suggests for evaluating fusion methods

1. **Report the deployable comparator.** The validation-selected single model is what fusion replaces in practice. Reporting only the hindsight-best model understates fusion's value whenever rankings churn.
2. **Report the equal-weight average of the same pool,** together with a pool-size sweep, so readers can see where learned weights begin to beat equal weights.
3. **Use the data the split leaves out.** The last four ETT months cost nothing to evaluate and test whether conclusions hold beyond one fixed test window.

6 Limitations

Base models were trained with a single seed, and only the fusor is averaged over seeds, so part of the base-model differences is training noise. Both datasets come from the same transformer stations and share a calendar. Results cover one horizon (96) and the six-model pool, not the paper's 13-model zoo. Base models were never retrained, so Protocol B tests refitting of the fusor only. Inputs that are constant within a window (46 windows on ETTh1 and 292 on ETTh2, under 4%) make the released feature extractor fail; they receive zero meta-features, matching how the released loader treats missing values. The paper lists 24 meta-features, while the released code computes 22 and omits the two cross-correlation features; I follow the code.

7 Next steps

The open question is when learned, sample-level weighting beats equal weighting. I would answer it in three steps. First, sweep the pool from 2 to 13 models, ordered by validation rank, and with deliberately weak members added. Second, extend the study to longer and more shift-prone series (Electricity, Weather, and my Lahore air-quality data, which has strong seasonal regime changes). Third, test whether simple regularization of the fusor toward uniform weights, or toward recent windows, keeps per-sample adaptivity where it helps and falls back to equal weights where it does not. This is the kind of evaluation-centred question about adaptive models that I hope to pursue in my PhD at Illinois.

Reproducibility. All code, logs and per-month results are in `github.com/imadkhan4478/timefuse-deployment-eval`. Base models were trained on a Colab T4 GPU (about 30 minutes). The full pipeline reruns from the authors' public repository with no change to their method.

References

Claeskens, G., Magnus, J. R., Vasnev, A. L., & Wang, W. (2016). The forecast combination puzzle: A simple theoretical explanation. *International Journal of Forecasting*, 32(3), 754–762.

Diebold, F. X., & Mariano, R. S. (1995). Comparing predictive accuracy. *Journal of Business & Economic Statistics*, 13(3), 253–263.

Harvey, D., Leybourne, S., & Newbold, P. (1997). Testing the equality of prediction mean squared errors. *International Journal of Forecasting*, 13(2), 281–291.

Künsch, H. R. (1989). The jackknife and the bootstrap for general stationary observations. *The Annals of Statistics*, 17(3), 1217–1241.

Liu, Z., Yang, Z., Lin, X., Qiu, R., Wei, T., Zhu, Y., Hamann, H., He, J., & Tong, H. (2025). Breaking silos: Adaptive model fusion unlocks better time series forecasting. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, PMLR 267.

Nie, Y., Nguyen, N. H., Sinthong, P., & Kalagnanam, J. (2023). A time series is worth 64 words: Long-term forecasting with transformers. In *ICLR*.

Stock, J. H., & Watson, M. W. (2004). Combination forecasts of output growth in a seven-country data set. *Journal of Forecasting*, 23(6), 405–430.

Tan, M., Merrill, M. A., Gupta, V., Althoff, T., & Hartvigsen, T. (2024). Are language models actually useful for time series forecasting? In *NeurIPS*.

Wang, S., Wu, H., Shi, X., Hu, T., Luo, H., Ma, L., Zhang, J. Y., & Zhou, J. (2024a). TimeMixer: Decomposable multiscale mixing for time series forecasting. In *ICLR*.

Wang, Y., Wu, H., Dong, J., Qin, G., Zhang, H., Liu, Y., Qiu, Y., Wang, J., & Long, M. (2024b). TimeXer: Empowering transformers for time series forecasting with exogenous variables. In *NeurIPS*.

Wu, H., Hu, T., Liu, Y., Zhou, H., Wang, J., & Long, M. (2023). TimesNet: Temporal 2D-variation modeling for general time series analysis. In *ICLR*.

Zeng, A., Chen, M., Zhang, L., & Xu, Q. (2023). Are transformers effective for time series forecasting? In *AAAI*.

Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., & Zhang, W. (2021). Informer: Beyond efficient transformer for long sequence time-series forecasting. In *AAAI*. (ETT datasets.)